

Projekt Angewandte Mathematik im Sommersemester 2026

Martin Hering-Bertram
Hochschule Bremen City University of Applied Sciences

11. Juni 2026

Zusammenfassung

Dieses Dokument spezifiziert die Anforderungen für das Projekt als Teilprüfung zum Portfolio im Fach Angewandte Mathematik für Medieninformatik. Es enthält die Bewertungsmaßstäbe und beispielhafte Projektthemen.

1 Motivation

Gegenstand des Projektes ist die eigenständige Recherche, Entwicklung und Analyse eines vertiefenden Themas angewandter Mathematik. Basierend auf einem Anwendungsszenario werden mathematische Werkzeuge zur Lösung eines Teilproblems ausgewählt und beschrieben. Numerische Beispiele werden ausgewertet und im Hinblick auf die Anforderungen des Anwendungsszenarios diskutiert. Das Projekt wird in einem technischen Bericht zusammen gefasst und in einem Kolloquium präsentiert. Lernziele sind eigenständiges wissenschaftliches Arbeiten, projektorientierte Teamarbeit und Vertiefung in einem Bereich der angewandten Mathematik. Für die Bewertung ist die Qualität der Ergebnisse wichtiger als die Quantität. Da das Projekt in Teamarbeit durchzuführen ist, trägt auch der Entwicklungsprozess und die Methodik zum Gesamteindruck bei.

2 Bewertungsmaßstab

Die Gewichtung des Projektes beträgt 50 Prozent der Gesamtnote. Das Projekt sollte in Gruppen von 2-4 Studierenden bearbeitet werden, wobei alle Teilnehmer:innen signifikant dazu beitragen und die Ergebnisse erläutern können. Gegenstand ist die Entwicklung

mathematischer Werkzeuge in einem Themenbereich des Moduls. Dazu gehören Recherche, Konzeption, Implementierung, Anwendung und Diskussion. Die Prüfungsleistung besteht aus einem Exposee, Zwischenpräsentationen, einem technischen Bericht und einem Kolloquium. Die Implementierung ist, wenn nicht anders vereinbart, in Python mit NumPy und SciPy vorzunehmen und zur Validierung mit einzureichen. Die Verwendung weiterer Programmiersprachen und Tools, z.B. Game Engines ist möglich und sollte im Exposee und im Bericht beschrieben sein.

Sollten Arbeiten Anderer oder aus anderen Modulen verwendet werden, ist die Urheberschaft im Detail erkennbar zu referenzieren. Dies gilt auch bei der Übernahme von Ideen oder von modifizierten Textbausteinen, welche ohne Quellenangabe unzulässig wären (Plagiate). Tools basierend auf künstlicher Intelligenz (KI) dürfen zur Verbesserung der Sprache verwendet werden, nicht jedoch zum Generieren von Inhalten. Jegliche Verwendung von KI-Tools ist zu deklarieren. Benutzt wird nur, was exklusiv für das Modul Angewandte Mathematik entwickelt wird. Die Ergebnisse sollten authentisch und reproduzierbar sein.

Gute wissenschaftliche Arbeiten enthalten neben allen verwendeten Quellen auch Literatur zu Grundlagen und Vorarbeiten mit einer knappen Zusammenfassung. Darauf basierend wird ein wissenschaftlicher Beitrag beschrieben (hier im Projekt muss dieser nicht neu sein, kann aber aus verschiedenen Methoden zusammen gesetzt sein). Es folgt eine Auswertung/Diskussion eigener Beispiele und Daten. Die Reproduzierbarkeit der Ergebnisse sollte gewährleistet sein. Im Bereich der Forschung gibt es weitere Richtlinien zur *Sicherung guter wissenschaftlicher Praxis*, z.B. formuliert von der Deutschen Forschungsgesellschaft (DFG), vgl. ¹.

Der technische Bericht sollte mittels LaTeX verfasst und wie folgt strukturiert sein:

- Abstract: Knappe Inhaltsangabe in 2-3 Sätzen (sollte man zum Schluss schreiben). Hier sollte keine Einführung ins Thema stehen.
- Motivation, Einführung, wissenschaftlicher Beitrag, Inhalt: Dies ist Gegenstand des ersten Abschnitts. Ein Unterabschnitt fasst den eigenständigen wissenschaftlichen Beitrag knapp zusammen, gefolgt von einer ganz knappen Inhaltsangabe (anstelle eines Inhaltsverzeichnisses). Das Projekt muss nicht zwangsläufig neue wissenschaftliche Techniken enthalten, aber die Beispiele sollten eigenständig erzeugt und auf Ebene der mathematischen Notation (Gleichungen, etc.) beschrieben werden.
- Verwandte Arbeiten / Grundlagen: Hier sollte man zwei bis drei Arbeiten beschreiben und darauf eingehen, wie diese zu den eigenen Ergebnissen stehen (also auch, welche Ideen man übernommen hat).

¹https://www.dfg.de/foerderung/grundlagen_rahmenbedingungen/gwp/

- Algorithmen/Implementierung: Dies ist der Hauptteil der Arbeit, welcher die eigenständige Konzeption und Entwicklung beschreibt. Dieser Teil kann aus mehreren Abschnitten bestehen.
- Numerische Beispiele: Dieser Abschnitt enthält Daten und Analysen und natürlich auch Abbildungen. Es empfiehlt sich aber auch, gleich zu Beginn des Papers Abbildungen einzusetzen, damit der Leser schneller mit dem Thema vertraut wird.
- Fazit und Ausblick: Probleme, die man im Rahmen der Arbeit nicht lösen konnte, kann man hier als zukünftige Arbeiten in Aussicht stellen. Auch die Möglichkeiten und Grenzen der vorgestellten Technik kann man nochmal nennen.
- Beiträge und Anerkennung: Hier sollten der eigenständige Beitrag zum Projekt und zum Bericht für jede Autor:in jeweils knapp aufgelistet sein. Auch externe Beiträge (z.B. Beratung durch andere Studierende) sollten hier gewürdigt werden. Der Einsatz von KI-Tools sollte ebenfalls genau beschrieben werden.
- Literatur. Hierfür bietet LaTeX eine eigene Umgebung. Wichtig ist, dass man hier im Wesentlichen nur referierte Veröffentlichungen und ggf. technische Berichte unterbringt. Dies sind Papers mit Autoren, Titel und Erscheinungsdatum, bei Zeitschriften mit Volume/Number, bei Konferenzen ggf. mit Konferenzdatum und Ort. Weiterhin sollte man Verlag und Seitenzahlen nennen, optional eine URL.

Weblinks, Blogs, etc. sind keine Literatur, da das Netz einem stetigen Wandel unterliegt und dessen Inhalte nicht einen gutachterlichen Prozess durchlaufen haben. Entweder richtet man ein eigenes Verzeichnis für Online-Quellen (mit Zugriffsdatum) ein, oder man bringt die URLs z.B. als Fußnoten im Text unter. Der Rechercheteil sollte, mindestens drei Arbeiten (wissenschaftliche oder Online-Quellen) enthalten.

Der Schreibstil für wissenschaftliche Arbeiten ist fokussiert und sachlich, ohne Umschweife (kein „Marketing“, keine Witze, keine „saloppe“ Sprache). Die „ich“-Form (und auch die Anrede, wie z.B. in Übungsaufgaben) sollte vermieden werden. Man kann die „wir“-Form verwenden, wenn man so schreibt, dass dies den Leser mit einschließt (kann man auch als alleiniger Autor, z.B. bei einer Thesis). In der Regel schreibt man fast alles im Präsens. Futur braucht man, wenn überhaupt, nur im letzten Abschnitt. Die Vergangenheitsform wird nur selten gebraucht, z.B. beim Datieren verwandter Arbeiten.

Wichtig ist auch noch, dass Abbildungen und Tabellen (Fließobjekte, welche durch LaTeX etwa am Seitenanfang platziert werden) mit Nummern und einer Aussagekräftigen Beschreibung versehen sind. Auf jedes dieser Objekte sollte an gegebener Stelle (mindestens einmal) aus dem Text verwiesen werden. Gleiches gilt für Literatur: Jeder Literaturangabe sollte man allerwenigstens einen Satz widmen und darauf verweisen. Wenn

man mit BibTeX arbeitet, erscheinen sowieso nur diejenigen Referenzen, die „Aufgerufen“ werden. Auf diese Weise kann man die selbe Literaturbasis für mehrere Arbeiten wiederverwerten.

Gleichungen werden von LaTeX automatisch in runden Klammern nummeriert. Man kann auf Gleichungen an anderer Stelle im Text über diese Nummern verweisen, muss das aber nicht tun. Die Nummerierung ist für den Austausch beim Schreiben mit mehreren Autoren und Gutachtern sehr nützlich.

Danksagung: Wer sich von anderen Personen (als die Autoren) beraten lässt, Ergebnisse gemeinsam mit anderen entwickelt oder teilt, sollte diese Personen auch benennen und sich für den geleisteten Beitrag bedanken. Dies schmälert den Wert der eigenen Arbeit nicht. Im Gegenteil: Es zeigt, dass man sich auch mit anderen Experten über die Lösungsansätze und eingesetzten Techniken ausgetauscht hat. Was bei Übungsaufgaben und Testaten nur eingeschränkt gilt (diese soll man ja selber lösen), wird hier im Projekt großzügiger gehandhabt, sofern ein beträchtlicher Teil auf eigenständiger Entwicklung und Analyse beruht.

Der technische Bericht sollte einerseits knapp und fokussiert geschrieben sein, andererseits aber alle Details, die zum Reproduzieren der Ergebnisse notwendig sind, enthalten. Eigene Ergebnisse und eine qualitative Analyse mit Bewertung im Hinblick auf die Anwendbarkeit sollten nicht fehlen. Insgesamt sollte der Bericht nicht länger als 12 Seiten sein.

Das Kolloquium besteht aus einem Kurzvortrag (10 min.) und einer Diskussion (5 min.). Die Implementierung ist zwecks Validierung mit abzugeben, ist i.d.R. aber nicht Gegenstand der Bewertung.

Kriterien für die Bewertung sind:

- Komplexität des Themas
- Dokumentation in Englisch (optional)
- Qualität der Ausarbeitung. Dies beinhaltet Recherche, fokussierte Darstellung in mathematischer Notation, technische Korrektheit, Auswahl der Beispiele, deren Bewertung und Reproduzierbarkeit, sowie Authentizität.
- Einhaltung formaler Kriterien (Sprache, Verweise auf Abbildungen und Literatur, Struktur, Vollständigkeit, etc.)
- Einhalten der Deadlines, Zwischenpräsentationen und Teamarbeit.
- Vortrag und Diskussion.

3 Projektbeispiele

Es sollte jede Gruppe (2-4 Studierende) ein eigenes Projektthema bearbeiten, welches z.B. einen Themenschwerpunkt des Moduls vertieft. Nachfolgend sind einige Projektbeispiele zur Ideenfindung und zur Orientierung beschrieben. Diese wurden z.T. bereits in vergangenen Semestern umgesetzt (vgl. Ausarbeitungen im Archiv) und dienen lediglich zur Orientierung. Es sollten eigenständige Themen gewählt werden, wobei Anknüpfungspunkte an die Projektbeispiele möglich und erwünscht sind. Die gewählten Themen sollten hinreichend verschieden sein.

Die Themenvorschläge werden nach Diskussion im Labor diskutiert und weiter konkretisiert. Danach ist ein Exposee zu erstellen, welches die genaue Spezifikation des Themas und Kriterien zur Qualitätssicherung enthält.

3.1 Simulation Mechanischer Systeme

Mechanische Systeme basieren auf dem Prinzip der Energieerhaltung (realisierbar durch die Euler-Lagrange-Gleichungen) und bei Kollision auf dem Prinzip der Impulserhaltung. Neben den linearen Größen von Geschwindigkeit, Kraft und Impuls existieren analoge Größen für Drehbewegungen: Winkelgeschwindigkeit, Drehmoment (engl. torque) und Drehimpuls (engl. angular momentum). Mechanische Systeme können mithilfe von Differentialgleichungen beschrieben und mittels numerischer Integration simuliert werden.

Thema 1: Dampfantrieb². Zu simulieren ist ein Dampfantrieb, bei dem eine konstante Kraft innerhalb eines Zylinders auf eine Pleuelstange wirkt und damit ein Rad antreibt. Bei der Rückwärtsbewegung kehrt sich die Dampfkraft um und schiebt den Kolben im Zylinder an seinen Ausgangspunkt zurück. Die Massenverteilung im Rad kann durch einen oder zwei punktförmige Massen dargestellt werden (alternativ durch einen Trägheits-Tensor). Das System ist mittels Euler-Lagrange-Gleichungen und Runge-Kutta-Integration zu simulieren. Die wirkenden Kräfte und Leistungen von Dampfmaschinen können so berechnet und mit der verfügbaren Literatur verglichen werden. Beispiel: Wie schnell beschleunigt ein Dampfzug mit gegebener Masse, welche Steigung würde er schaffen. Die Parameter wie Dampfkraft, Anzahl der Zylinder, etc. können manuell eingestellt werden. Unbekannte Parameter kann man ggf. mit Hilfe der Simulation herausfinden.

²<https://de.wikipedia.org/wiki/Dampfmaschine>

Thema 2: Kolbenmotor³. Dieses Thema ist vom physikalischen Aufbau sehr ähnlich zu Thema 1. Ein wesentlicher Unterschied ist jedoch, dass die antreibende Kraft nicht kontinuierlich wirkt, sondern bei jeder Zündung durch einen Impuls (“Masse mal Geschwindigkeit”) übertragen wird. Die Simulation ist ebenfalls mithilfe der Euler-Lagrange-Gleichungen und Runge-Kutta-Integration zu implementieren. Mit den geometrischen Parametern des Motors kann man experimentieren, um die Leistung zu optimieren. Weiterhin kann man das Drehmoment berechnen, welches der Motor bei einem Übertragungsfaktor durch das Getriebe erzeugt.

Thema 3: Physik-Engines. Mithilfe von Physik- oder Dynamics-Engines lassen sich Mehrkörpersysteme simulieren. Anhand von Beispielen soll dargestellt werden, wie mechanische Starrkörper-Systeme modelliert und in eine Dynamics-Engine eingegeben werden können. Bei diesem Thema wäre zuerst zu recherchieren, mit welcher Engine es sich lohnt zu arbeiten und welche Kenntnisse (z.B. C++ bei Verwendung der Open Dynamics Engine (ODE)) notwendig sind. Nach Auswahl und Installation der Engine muss man sich zuerst mit der Funktionalität vertraut machen. Von Interesse sind Starrkörper, welche durch Drehgelenke miteinander verbunden sind (vgl. Themen 1+2), sowie Kollisionen von Körpern. Anhand von Skizzen und mathematischer Beschreibung der Parameter eines oder mehrerer Systeme soll exemplarisch gezeigt werden, wie man mit der Engine eine Simulation erstellt und auswertet.

Thema 4: 3D-Achterbahn. Zu Beginn der Laboraufgaben haben wir eine 2D-Achterbahn simuliert. Ziel dieses Themas ist es, eine zweigleisige 3D-Bahnkurve zu erstellen und darauf bewegliche Objekte, welche Gravitationskräften und Beschleunigungs-kräften durch Motoren unterliegen, zu simulieren. Die Implementierung erfolgt in Matlab, unter Zuhilfenahme von Toolboxen, z.B. zur Erstellung der Kurve mittels Splines. Die Kurve muss in Bogenlängenparametrisierung vorliegen. hierfür diskretisiert man die Mittelpunktskurve in einen Polygonzug, welcher chordal parametrisiert wird. Die beiden Schienen lassen sich mittels Frenet-Frame aus der analytischen Darstellung der Kurve berechnen und an den selben Stellen, wie die Mittelpunktskurve abtasten.

Thema 5: Kollisionsbehandlung. Wie lassen sich Geschwindigkeit und Winkelgeschwindigkeit polygonaler 2D-Objekte mit Masseverteilung nach einer Kollision basierend auf der Impulserhaltung und Drehimpulserhaltung berechnen? Zu betrachten ist zunächst die Kollision eines Objektes mit einer Geraden (bzw. Ebene), danach die Kollision zweier Objekte. Um die Kollisionsbehandlung zu veranschaulichen, sollte eine Simulation in Matlab erstellt werden. Die Verallgemeinerung auf 3D-Objekte ist optional. Die Erkennung der Kollision ist nicht Gegenstand dieses Themas. Die Beispiele können so gewählt sein, dass die gegenseitige Lage der Objekte zum Kollisionszeitpunkt bekannt ist. die Bewegung der Objekte vor der Kollision kann aufgrund der Geschwindigkeiten

³<https://de.wikipedia.org/wiki/Hubkolbenmotor>

und Winkelgeschwindigkeiten rückwärts berechnet werden.

3.2 Inverse Kinematik

Die inverse Kinematik stellt eine Anwendung nichtlinearer Systeme (vgl. nächsten Abschnitt). Die Lösung kann mit dem Newton-Raphson-Verfahren erfolgen. Dies kann man mit weiteren Techniken, wie Fehlerquadrate-Minimierung und Lagrange-Multiplikatoren kombinieren. Eine robuste Variante für solche Probleme stellt der Levenberg-Marquardt-Algorithmus ⁴ [3, 4] dar.

Thema 6: Simulation einer Gehbewegung. Gegenstand dieses Themas ist die Simulation der Gehbewegung einer menschenähnlichen Figur in Matlab. Hierfür werden die Beine als zweidimensionale kinematische Kette (in Seitenansicht) modelliert. Es sollten drei bis vier Gelenke (Hüfte, Knie, Ferse, Fußballen) verwendet werden. Das Gelenk im Fußballen kann so eingestellt werden, dass der Vorderfuß bei Bodenkontakt parallel zum Boden aufliegt. Die Position von Hüfte und Füßen kann durch eine parametrische Kurve festgelegt werden. Für die Darstellung kann ein dreidimensionales Modell verwendet werden. Optional kann auch noch der Oberkörper modelliert werden. Die Übertragung des Modells in ein anderes Tool zur Darstellung (Blender, Unity, etc.) ist ebenfalls optional. Wichtig ist, dass die mathematische Modellierung im Detail beschrieben wird und dass es Mechanismen zur Auswertung gibt, z.B. bezüglich des Gleichgewichts.

Thema 7: Simulation eines Vierfüßlers. Der Unterschied zum vorangehenden Thema ist, dass bei einem Vierfüßler (Pferd, Hund, etc.) jeweils zwei Beinpaare mit ggf. unterschiedlichen Parametern koordiniert werden müssen. Im einfachsten Fall berühren immer zwei diagonal gegenüber liegende Füße den Boden, derart dass es nicht zu einem Ungleichgewicht kommt. Es können weitere Gelenke modelliert werden. Eine sinnvolle Erweiterung wären unterschiedliche Bewegungsmuster etwa Schritt, Trab und Galopp für ebenes Gelände. Auch hier erfolgt die Implementierung in Matlab mit optionaler Übertragung in andere Tools zur Darstellung und Analyse.

Thema 8: 3D-Roboter-Spinne. Im Gegensatz zum Zwei- und Vierfüßler besitzt die Spinne auch bei unwegsamem Gelände stets einen stabilen Stand. hierfür genügen sechs Beine, von denen stets drei den Boden berühren müssen (zwei weitere kann man optional als Fühler einsetzen). Ziel dieses Projektes ist es, einen Algorithmus zur Steuerung der Spinne in einem Gelände mit Oberflächenprofil (Graphfläche oder Bitmap als Höhenprofil) zu entwickeln und zu testen. Die Beine können relativ einfach, z.B. mit jeweils zwei Gelenken modelliert werden.

⁴<https://de.wikipedia.org/wiki/Levenberg-Marquardt-Algorithmus>

Thema 9: Roboterarm. Hier ist ein fest installierter Industrieroboter mit rotierenden und prismatischen Gelenken zu modellieren. Der Roboter kann dann z.B. einen Fräskopf bewegen, welcher aus einem virtuellen Material eine Freiformfläche ausfräst, indem er sich entlang einer Offset-Fläche bewegt. Der Offset ist dabei mit dem Radius des Fräskopfes identisch. Alternativ kann der Roboter andere Aufgaben erledigen, z.B. virtuelle Bauteile je nach Farbe in Schubladen einsortieren.

3.3 Nichtlineare Systeme

Neben der inversen Kinematik existieren noch weitere Anwendungen, bei denen ein Nichtlineares Gleichungssystem zu lösen ist. Auch hier kann man mit der Newton-Raphson-Iteration oder den oben beschriebenen Varianten arbeiten.

Thema 10: 3D-Tracking. Gegeben ist ein „Target“, bestehend aus mehreren (4-8) Punkten (z.B. Infrarot-Marker), die in einem lokalen Koordinatensystem fest verbunden sind. Die Position dieser Punkte kann man so wählen, dass ihre gegenseitigen Abstände möglichst unterschiedlich groß sind. Die Marker werden von einer Kamera aufgenommen, die im Ursprung platziert und in Richtung der negativen z-Achse ausgerichtet ist. Die Umrechnung der Weltkoordinaten x_w, y_w, z_w in Kamerakoordinaten x_c, y_c wird durch folgende Projektion berechnet, wobei a und b Kamera-spezifische Konstanten sind:

$$[x_c, y_c, z_c]^T = \left[\frac{a \cdot x_w}{z_w}, \frac{b \cdot y_w}{z_w}, 1 \right]^T \quad (1)$$

Bevor das Target aufgenommen wird, wird es mit einer affinen Transformation (Rotation und Translation) im Weltsystem platziert:

$$[x_w, y_w, z_w]^T = R_\alpha R_\beta R_\gamma [x_l, y_l, z_l]^T + [t_1, t_2, t_3]^T. \quad (2)$$

Hierbei sind α, β, γ Euler-Winkel und t ist ein Translationsvektor. Diese sechs Parameter sind nun Gegenstand der Berechnung: Aus dem projizierten Bild der Marker in Kamerakoordinaten soll mithilfe der bekannten lokalen Koordinaten x_l, y_l, z_l die passende affine Transformation ermittelt werden. Mit einer ersten Schätzung für R und t berechnet man die Projektionen der Marker und vergleicht diese mit den zu erreichenden Koordinaten. Diese Projektionsfehler sind zu minimieren, indem man die sechs Parameter der affinen Transformation iterativ verbessert.

Da dieses Problem sehr komplex ist, sollte man es zuerst nur in 2D lösen, d.h. ohne y -Koordinaten und mit nur einem Rotationswinkel α .

Thema 11: 3D-Rekonstruktion. Hier soll die räumliche Lage eines Objektes bestimmt werden, welches von zwei Kameras mit der in Gleichung (1) beschriebenen Projektion

aufgenommen wird. Die erste Kamera ist, wie in obigem Thema, im Ursprung platziert. Die zweite ist dazu mit einer bekannten affinen Transformation versetzt. Nun sollen mehrere Punkte in Weltkoordinaten aus ihren Kamerakoordinaten in beiden Ansichten rekonstruiert werden. Dieses Problem kann alternativ geometrisch mit Methoden der Epipolar-geometrie⁵ gelöst werden.

Thema 12: Verschneiden von Parameterflächen. Für ein Raytracing-Verfahren oder für Kollisionserkennung ist eine Parameterfläche $f(s, t)$ mit einem parametrischen Strahl $g(u) = \mathbf{p} + u\mathbf{v}$ zu verschneiden. Zuerst schätzt man die Parameter s, t, u durch Anfangswerte. Innerhalb der Iteration bestimmt man nun den Schnittpunkt des Strahls mit der Tangentialebene in $f(s, t)$. Die s, t -Koordinaten des Schnittpunktes nimmt man als Korrekturen für die Flächenparameter und wiederholt die Berechnung mit den neuen Parametern. Dies entspricht dem Newton-Raphson-Verfahren. Um alle Schnittpunkte zu bestimmen, muss man den Algorithmus mit verschiedenen Startwerten für die Flächenparameter ausführen und nur die Ergebnisse zurückgeben, bei denen Konvergenz erfolgt ist.

3.4 Polygonale Objekte

Thema 13: Konvexe Hülle. In Matlab existiert die Funktion `convhull()` zur Berechnung der konvexen Hülle einer Punktmenge. Es wurde jedoch noch nicht untersucht, welche Algorithmen es tatsächlich gibt. Neben dem Giftwrapping-Algorithmus, der die Punkte quasi in „Geschenkpapier“ verpackt, gibt es noch weitere Ansätze, die man bezüglich ihrer Laufzeit untersuchen und mit der Matlab-Funktion vergleichen kann. Hierfür sollten Punktemengen verschiedener Größe und Struktur in 2D und 3D als Eingabe dienen.

Thema 14: Minkowski-Summe. Für zwei konvexe Objekte ist die Berechnung der Minkowski-Summe mit moderatem Aufwand realisierbar. In diesem Projekt wird angenommen, dass eines der Objekte nicht konvex ist, z.B. soll ein konvexer Roboter gegen ein nicht-konvexes Polygon getestet werden. Für diesen Fall kann man das nicht-konvexe Objekt in konvexe Polygone, z.B. Dreiecke zerlegen und deren Minkowski-Summen mit dem konvexen Roboter separat berechnen. Zum Schluss werden die resultierenden Polygone rekursiv wieder zusammengefügt, indem man sie miteinander verschneidet und die „inneren“ Polygonzüge entfernt. Die Erweiterung in drei Dimensionen ist optional.

Thema 15: 3D-Triangulierung. Gegeben ist eine Punktwolke, z.B. als Ergebnis eines Laser-Scans. Eine der ersten Forschergruppen, welche solche Daten zur Flächenrekonstruktion Verfügung gestellt hat ist die Computergraphik-Gruppe der Stanford University⁶ Mittlerweile gibt es im Netz viele Datensätze. Ein vergleichsweise einfacher Algo-

⁵<https://de.wikipedia.org/wiki/Epipolargeometrie>

⁶<http://graphics.stanford.edu/data/3Dscanrep/>

rithmus basiert auf der Idee der sogenannten Alpha-Shapes⁷: Vereinfachend bewegt man eine virtuelle Kugel mit Radius α von „außen“ an der Punktwolke entlang. Wird der Radius groß genug gewählt, so entsteht eine triangulierte Fläche, welche aus solchen Dreiecken besteht, die zu klein sind um die Kugel hindurchzulassen. Diesen oder einen ähnlichen Algorithmus gilt es zu entwickeln und die Triangulierung auf Überschneidungs- und Lückenfreiheit zu testen.

Thema 16: Voronoi-Diagramm / Delaunay-Triangulierung. Die Delaunay-Triangulierung verbindet eine Punktmenge in der Ebene durch Dreiecke, derart dass die konvexe Hülle komplett mit Dreiecken ausgefüllt ist. Sie besitzt die Eigenschaft, dass der Umkreis jedes Dreiecks keinen anderen Datenpunkt enthält. Man kann die Delaunay-Triangulierung durch inkrementelles Einfügen der Punkte in eine bestehende Triangulierung berechnen.

In diesem Projekt ist ein Algorithmus für die Delaunay-Triangulierung zu entwickeln und bezüglich des Laufzeitverhaltens mit der entsprechenden Matlab-Funktion zu vergleichen. Eine mögliche Erweiterung ist die Konstruktion des Voronoi-Diagramms als „dualer Graph“ der Triangulierung.

Thema 17: Kollisionserkennung. Für das Testen von Kollisionen zwischen dreidimensionalen triangulierten Objekten sollen verschiedenartige Algorithmen entwickelt werden. Mögliche Ansätze sind Bounding-Boxes, Bounding-Spheres, sowie die Minkowski-Summe der konvexen Hüllen beider Objekte. Die Tests sind bezüglich Rechenzeit und Genauigkeit zu vergleichen. Optional kann man die Kollision durch gegenseitiges Verschneiden der Dreiecke exakt berechnen (mit sehr viel Aufwand).

3.5 Kurven und Flächen

Kurven und Flächen sind ein zentrales Thema der angewandten Mathematik. Entsprechend vielseitig sind die Themen in diesem Bereich.

Thema 18: Moving Least Squares. Die Methode der Moving Least Squares⁸ kann verwendet werden, um eine Punktmenge, z.B. als Ergebnis aus einem Laserscan, durch eine kontinuierliche Fläche zu approximieren. Um die Funktion an einem Punkt $\mathbf{p}$ zu ermitteln, gewichtet man die Datenpunkte mit einer Gauß-Funktion $\theta(\mathbf{x} - \mathbf{p})$, so dass im Wesentlichen die Punkte in einer direkten Umgebung in die nachfolgende Berechnung eingehen. Die Funktion ist dann eine Ebene, die mithilfe der Hauptachsentransformation (eng. principal component analysis, PCA) errechnet wird. Da sich die Ebene mit dem Punkt $\mathbf{p}$ ändert, entsteht eine kontinuierliche Fläche.

⁷https://en.wikipedia.org/wiki/Alpha_shape

⁸https://en.wikipedia.org/wiki/Moving_least_squares

Thema 19: Radiale Basisfunktionen. Dieses Projekt beschäftigt sich mit Scattered-Data-Interpolation. Gegeben sind Ordinaten f_i an beliebig in der Ebene verteilten Stützpunkten $[s_i, t_i]^T$. Anstelle einer polynomialen Basis wählt man zum Interpolieren eine Menge aus radialen Basisfunktionen⁹. Diese enthält im einfachsten Fall zu jedem Stützpunkt eine Gauß-Funktion $\phi_i(s, t)$, die an diesem Stützpunkt maximal ist. Das Interpolationsproblem ist, wie schon sooft durch ein Gleichungssystem $Ac = f$ nach den Koeffizienten zu lösen, wobei $A_{ij} = \phi_j(s_i, t_i)$ ähnlich der Vandermonde-Matrix die Auswertungen der Basisfunktionen an den Stützstellen enthält. Neben der Gauß-Basis gibt es weitere Möglichkeiten (Multiquadrik und inverse Multiquadrik).

Thema 20: Editieren von Kurven und Flächen. Dieses Projekt hat zum Gegenstand, einen Editor für Splinekurven und -flächen zu entwickeln, bei welchem die Kontrollpunkte interaktiv eingegeben werden können. Optional können verschiedenartige Flächen (Interpolation, Bézier, B-Spline) verwendet werden.

Thema 21: Subdivision-Algorithmen. Kurven lassen sich am effizientesten durch rekursives Unterteilen eines Kontrollpolygons berechnen und zeichnen. Sowohl interpolierende Kurven [1], als auch Splinekurven lassen sich auf diese Weise erzeugen. Einige Schemata findet man am Ende von Kapitel 4 der Vorlesung. Gegenstand des Projektes ist es, verschiedene Schemata zur Kurvengenerierung zu implementieren und zu vergleichen.

Thema 22: Implizite Flächen. Flächen dieser Form erhält man als Nullmenge eines Skalarfeldes $f(x, y, z) = 0$. Einfache Flächentypen wie Zylinder, Kegel, Kugeln, Ellipsoide und Paraboloiden lassen sich als Quadriken erzeugen. Schwierig ist nur, wie man diese Flächen graphisch darstellt. In der Datenvisualisierung hat sich der Marching-Cubes-Algorithmus¹⁰ etabliert, welcher den Raum in Würfel (engl. cubes) unterteilt und dort bei Bedarf eine Triangulierung aus einer Tabelle einsetzt. Im Rahmen dieses Projektes soll ein möglichst einfacher Ansatz entwickelt werden, um implizite Flächen durch Meshes aus Dreiecken oder Vierecken darzustellen.

Thema 23: Trimmflächen. Solche Flächen entstehen, indem man einen Bereich aus dem Definitionsgebiet einer Parameterfläche $f(s, t)$ durch eine darin befindliche geschlossene Kurve $[s(u), t(u)]^T$ „ausschneidet“ und nur den Flächenteil zeichnet, dessen Parameter innerhalb der geschlossenen Kurve liegen. Im Rahmen dieses Themas ist ein Algorithmus zur graphischen Darstellung von Trimmflächen zu entwickeln und an verschiedenen Beispielen zu testen.

Thema 24: Sweepflächen und abwickelbare Flächen. Bewegt man eine Kurve entlang einer anderen, möglichst senkrecht dazu verlaufenden Kurve, so entsteht eine Sweepfläche. Abwickelbare Flächen sind solche, die man ohne eine tangentialen Dehnung oder

⁹https://en.wikipedia.org/wiki/Radial_basis_function

¹⁰https://de.wikipedia.org/wiki/Marching_Cubes

Stauchung in der Ebene auslegen kann, wie z.B. die zugeschnittenen Teile eines Kleidungsstücks. Solche Flächen sind Spezialfälle von Parameterflächen und sollen in diesem Thema anhand von Beispielen untersucht und klassifiziert werden. Optional können beliebige Parameterflächen auf ihre Abwickelbarkeit untersucht werden.

Thema 25: Bézier-Dreiecke. Neben den Tensorprodukt-Flächen existiert ein weiterer Flächentyp für Polynome, welcher auch eine Darstellung mit Bernstein-Polynomen besitzt ¹¹ [2]. Dreiecksflächen verhalten sich isotrop, da sie nur einen Polynomgrad besitzen (statt zwei, wie bei Tensorprodukten). Hier sollen die Eigenschaften dieser Flächen recherchiert und anhand von Beispielen visualisiert werden. Von besonderem Interesse ist, wie man mehrere Bézier-Dreiecke zu einer C^1 -stetigen Splinefläche zusammensetzen kann.

Thema 26: Free-Form-Deformations. Eine FFD¹² ist eine Abbildung des Raumes (oder der Ebene) in sich selbst, bei der es in der Regel zu Verzerrungen kommt. Erzeugen kann man eine FFD z.B. durch eine Splinefläche mit Kontrollpunkten in der Ebene, wobei das Definitionsgebiet und das Bild dieser Funktion identisch sind. Bewegt man nun eine gerade Linie durch die FFD, so schlängelt sich diese wenn man die Kontrollpunkte der einzelnen Spalten gegeneinander verschiebt. Auch in 3D kann man FFDs z.B. als Tensorprodukt aus Basis-Splines in drei Parametern auf einem Würfelgitter von Kontrollpunkten erzeugen. Mit FFDs lassen sich spektakuläre Effekte in der Computer-Animation auf recht einfache Weise erzeugen.

3.6 Computer-Algebra

Computeralgebrasysteme (CAS) erleichtern die Arbeit bei komplexen Problemen, etwa beim Lösen nichtlinearer Gleichungssysteme oder beim Integrieren von Differentialgleichungen. Die folgenden CAS können im Hinblick auf eine der o.g. Anwendungen untersucht werden:

Thema 27: Wolfram-Alpha. <https://www.wolframalpha.com/>

Thema 28: Sage. <https://www.sagemath.org/>

Thema 29: Maxima. <http://maxima.sourceforge.net/>

Thema 30: Symbolic Math (Matlab).

Von besonderen Interesse ist neben dem Auffinden von Lösungen, wie die zugrunde liegenden Probleme zum besseren Verständnis visualisiert werden können. Als Testfälle wählt man ein oder mehrere komplexe Probleme, z.B. Teilprobleme aus einem der obi-

¹¹<https://de.wikipedia.org/wiki/Bezierfl%C3%A4che>

¹²<https://de.wikipedia.org/wiki/>

gen Abschnitte und zeigt, wie diese in das CAS eingegeben und die Lösung visualisiert werden kann. Insbesondere das Lösen nichtlinearer Probleme, vgl. Abschnitt 2.3, oder die Integration von Differentialgleichungen können als Schwerpunkt ausgewählt werden. Mit Ausnahme von Thema 30 muss in diesem Bereich nicht unbedingt mit Matlab gearbeitet werden. Matlab kann bei Bedarf zum Vergleich herangezogen werden.

Literatur

- [1] Nira Dyn, David Levin and John A. Gregory, *A 4-point interpolatory subdivision scheme for curve design*, Computer Aided Geometric Design, vol. 4, no. 4, Elsevier, 1987, pp. 257-268.
- [2] Gerald Farin, *Curves and Surfaces for CAGD. A practical guide*. 5. Auflage, Academic Press, San Diego 2002, ISBN 1-55860-737-4
- [3] Kenneth Levenberg *A Method for the Solution of Certain Problems in Least Squares*, Quarterly of Applied Mathematics, vol. 2, no. 2, American Mathematical Society (AMS), 1944, pp. 164-168.
<https://www.ams.org/journals/qam/1944-02-02/home.html>
- [4] Donald W. Marquardt, *An Algorithm for Least-Squares Estimation of Non-linear Parameters*, SIAM Journal on Applied Mathematics, vol. 11, no. 2, Society for Industrial and Applied Mathematics (SIAM), 1963, pp. 431-441.
<https://epubs.siam.org/doi/abs/10.1137/0111030>